ARALert System Technical & Security Audit

Architecture, Data Safety, RBAC, RA 10173 Compliance & Performance Report

System: ARALert Full-Stack Platform
Date: October 8, 2026
Version: 2.0 (Production Architecture)
Status: Validated & Audited

1. Executive Summary

This audit provides an exhaustive evaluation of the **ARALert** platform across its entire architecture: Django REST Framework backend, PostgreSQL/SQLite database models, Google OAuth 2.0 authentication, and React 19/TypeScript frontend. The report answers all critical questions concerning data safety, user permissions, edge case resilience, Philippine regulatory compliance (Data Privacy Act of 2012 / RA 10173), rate limiting, and system latency.

2. Data Safety, Storage & Access Control (RBAC Matrix)

Data at Rest & Identity Security:

- **Staff Accounts (SuperAdmin, Admin, Principal, Faculty, Tutor):** Protected with zero plaintext or hashed passwords stored in the database. Authentication is 100% delegated to Google OAuth 2.0 via verified OpenID Connect Subject IDs (`google_sub`).
- **Student Accounts:** Authenticated using Full Name (First Name, Middle Name, Last Name) and 12-digit Learner Reference Number (LRN).
- **Database Security:** Production is configured for PostgreSQL with SSL encryption required (`ssl_require=True` in `settings.py`). SQLite is isolated to local development environments.
- **Session Lifecycle:** JWT access tokens (30-minute lifespan) and refresh tokens are stored in browser `sessionStorage`, which automatically flushes upon tab or browser termination.

Role-Based Access Control (RBAC) Matrix

ROLE	USER MANAGEMENT	ACADEMIC RECORDS / GRADES	ATTENDANCE RECORDS	ANALYTICS / REPORTS	AUDIT TRAIL
SuperAdmin	FULL CRUD (ALL ROLES)	GLOBAL READ / AUDIT	GLOBAL READ / AUDIT	SYSTEM-WIDE	FULL GLOBAL ACCESS
School Admin	SCHOOL STAFF & STUDENTS	SCHOOL-WIDE OVERSIGHT	SCHOOL-WIDE OVERSIGHT	TRANSITION & SCHOOL KPIS	SCHOOL ACTIVITY LOGS
Principal	READ-ONLY (SCHOOL STAFF)	READ-ONLY (SCHOOL GRADES)	READ-ONLY (ATTENDANCE)	EXECUTIVE DASHBOARDS	READ-ONLY (LIMITED)
Faculty	NONE	SUBJECT-SPECIFIC PERFORMANCE	READ-ONLY (CLASS ATTENDANCE)	SUBJECT ANALYTICS	NONE
Tutor	EDIT REQUEST SUBMISSION ONLY	ASSIGNED STUDENTS ONLY	ASSIGNED STUDENTS ONLY	TUTOR COHORT METRICS	NONE
Student	NONE (SELF PROFILE ONLY)	SELF GRADES ONLY	SELF ATTENDANCE ONLY	SELF PROGRESS ANALYTICS	NONE

3. Data Integrity & Ingestion Failures (Handling Bad Data)

- **Serializer Regex & Sanitization:** All user and academic inputs undergo strict pattern verification (`NAME_PATTERN`, `PHONE_PATTERN`, `SECTION_PATTERN`, `DANGEROUS_INJECTION_PATTERN` in `api/users/serializers.py`). HTML tags (`<script>`, `<iframe>`) are automatically stripped by `sanitize_input_text()`.
- **Database Integrity Constraints:** Unique constraints on `email`, `lrn`, `google_sub`, `(student, school_year)`, and `(student, date)` attendance records prevent duplicate or conflicting data at the storage layer.
- **Cascading Protection:** Academic tables enforce `on_delete=models.PROTECT`, preventing accidental deletion of students with active records.
- **Fail-Safe Error Handler:** Unhandled exceptions trigger `custom_exception_handler` in `utilities/exceptions.py`, logging the trace to server logs and returning a standardized `500 Internal Server Error` (`{"detail": "An unexpected error occurred."}`) rather than exposing backend code structures.

4. Backend Access Control & Role Permissions Audit

- **Permissions Layer:** Django REST Framework permission classes (`IsSuperAdmin`, `IsAdmin`, `IsTutor`, `IsStudent`, `IsPrincipal`, `IsFaculty`, and `role_required(...)`) protect all API views.
- **Object-Level Scoping:** In individual detail views (e.g. updating grades or activities), explicit checks ensure that a tutor can only view or mutate records belonging to students assigned to their ID.
- **Middleware Cleanup:** `RoleMiddleware` in `backend/middleware/role_middleware.py` was identified as an empty pass-through and can be replaced with active role header validation or retired.

5. API Keys, Secrets & Frontend Exposure Audit

> **CRITICAL AUDIT FINDING IN `frontend/.env`:**
> The local `frontend/.env` file contained backend configuration keys: `SECRET_KEY`, `SMTP_EMAIL`, `SMTP_PASSWORD` (Google App Password), and `DB_PASSWORD`.

- **Client Bundle Analysis:** Vite strictly bundles only environment variables prefixed with `VITE_` into compiled assets (e.g., `VITE_GOOGLE_CLIENT_ID` and `VITE_API_URL`). Raw database and SMTP passwords are *not* included in the compiled JavaScript bundle.
- **Exposure Vector:** Keeping backend credentials inside the `frontend/` directory creates a risk if the repository is committed to GitHub or deployed to static hosts (Vercel, Netlify).
- **Remediation:** Remove all non-`VITE_` variables from `frontend/.env`. All secrets must remain exclusively in `backend/.env`.

6. Stress Testing "Ugly Scenarios" & Edge Cases

UGLY SCENARIO	CURRENT SYSTEM REACTION	HARDENED RESILIENCE IMPLEMENTATION
Wrong Password / Invalid Login	Staff: Google OAuth rejects unregistered emails with `403 Forbidden`. Students: Name/LRN mismatch returns `400 Bad Request`. Brute Force: 10 failed attempts trigger `429 Too Many Requests` (15-min IP lock).	Add an active countdown cooldown banner in the frontend UI when a `429` status code is received.
No Internet / Network Loss	Axios triggers an unhandled network error. If a token refresh is attempted while offline, `axios.ts` calls `handleForceLogout()`, prematurely terminating the user's session.	Distinguish true `401 Unauthorized` from offline network errors (`!window.navigator.onLine` / `ERR_NETWORK`). Display a persistent "Offline — Reconnecting..." banner without clearing session tokens.
Unexpected / Malicious Input	Django ORM parameterized SQL blocks SQL injection. Regex filters block `<script>` and SQL keywords.	Enforce frontend `maxLength` on all form fields and configure backend maximum JSON payload size limits.

7. Error Logging, Backup & Basic Monitoring

- **Existing Security Audit Trail:** The `AuditLog` table logs major security events (`LOGIN_SUCCESS`, `LOGOUT_SUCCESS`, `STUDENT_LOGIN_FAILED`, `PROFILE_UPDATED`, `ALLOWED_EMAIL_ADDED`) along with client IP address, user agent, and timestamp.
- **Monitoring Enhancements to Implement:**
  1. **Rotating Disk Logging:** Configure `logging.handlers.RotatingFileHandler` in `settings.py` writing to `backend/logs/aralert_errors.log` (5MB max file size, 5 backups).
  2. **Automated Error Webhook:** Dispatch an alert (via Email or Webhook) if 500 error counts spike.
  3. **Health Check Endpoint:** Expose an unauthenticated `/api/health/` route for external uptime pings (e.g., UptimeRobot).

8. API Request Efficiency & Redundant Call Elimination

- **Redundant Profile Mounting:** `useAuth.tsx` called `authService.getProfile()` on every page mount, even when user data was already stored in `sessionStorage`.

- **Sequential Pagination Loops:** In `academicService.ts` (`getGrades` and `getAttendanceHistory`), a `while (next)` loop sequentially requested every page of records across the network, causing significant browser stalls.
- **Remediation:** Rely on cached session state on mount and use server-side pagination (`page`, `page_size`) in table components rather than fetching full datasets upfront.

9. Rate Limiting & Usage Limits Architecture

```
# Recommended REST_FRAMEWORK Throttling Configuration in backend/config/settings.py REST_FRAMEWORK = {
'DEFAULT_THROTTLE_CLASSES': [ 'rest_framework.throttling.AnonRateThrottle',
'rest_framework.throttling.UserRateThrottle', 'rest_framework.throttling.ScopedRateThrottle', ],
'DEFAULT_THROTTLE_RATES': { 'anon': '60/minute', 'user': '300/minute', 'login': '10/minute', 'scan':
'30/minute', 'export': '10/minute', } }
```

10. Retry Policies & Infinite Loop Protections

- **Refresh Token Loop Prevention:** In `axios.ts`, `originalRequest._retry = true` prevents recursive 401 refresh loops.
- **Exponential Backoff:** Network retry policies must cap at a **maximum of 3 retries** with exponential delays (1s, 2s, 4s).
- **Idempotency Rule:** Retries must strictly apply only to idempotent requests (`GET`, `HEAD`), never to data-mutating `POST` or `DELETE` calls.

11. Understanding & Implementing "Catching" (Error Handling)

> **Technical Explanation of "Catching":**
> In software engineering, **"Catching"** refers to structured Exception Handling (`try ... catch` in TypeScript and `try ... except` in Python). When an unexpected failure occurs (network timeout, database deadlock, null reference), the environment throws an **Error**. If uncaught, the entire application crashes or produces a blank screen. "Catching" intercepts the error, writes diagnostics to logs, and provides a graceful user experience (such as an informative toast message) without crashing the interface.

12. Abnormal Usage & Security Alerting

- **Spike Monitoring:** Alert system administrators if 500 error counts exceed 20 events in 5 minutes.
- **Brute Force Alerts:** Notify administrators when an IP generates >15 failed authentication attempts within 10 minutes.
- **Traffic Anomalies:** Flag IPs generating >500 requests/minute.

13. Endpoint Vulnerabilities & Abuse Vectors (Bot Protections)

ENDPOINT	VULNERABILITY / ABUSE VECTOR	PROTECTIVE COUNTERMEASURE
`/api/auth/student-login/`	12-digit LRN enumeration and brute forcing	IP-based rate limit (10/min) + Cloudflare Turnstile / CAPTCHA on repeat failure
`/api/academics/attendance/scan/`	QR attendance token flooding and replay	Single-use session tokens + rate limit (30 scans/min)
`/api/academics/archive/.../export/`	Heavy PDF/Excel export CPU/RAM exhaustion	Asynchronous background exports + rate limit (10 exports/min)
`/api/time/`	Unauthenticated DDoS flood target	Anonymous throttling (60/min)

14. Operational API & Infrastructure Cost Estimation at Scale

COMPONENT	100 – 500 STUDENTS	1,000 – 5,000 STUDENTS	10,000+ STUDENTS	NOTES & LIMITS
Google OAuth	$0 / MO	$0 / MO	$0 / MO	Google Identity Services authentication is completely free.
Frontend Hosting	$0 / MO (VERCEL)	$0 / MO (FREE TIER)	$20 / MO (VERCEL PRO)	Free tier covers up to 100GB monthly bandwidth.
Backend Server	$5 – $7 / MO	$15 – $25 / MO	$50 – $80 / MO	Render / Railway / AWS Lightsail (Gunicorn workers).
PostgreSQL DB	$0 / MO (SUPABASE/NEON)	$15 – $25 / MO	$50 – $100 / MO	Managed PostgreSQL with daily backups and connection pooling.
Transactional Email	$0 / MO	$0 – $15 / MO	$15 – $35 / MO	Resend / Brevo / AWS SES (Gmail free SMTP caps at 500/day).
Total Monthly Cost	**~$5 – $7 / mo**	**~$30 – $65 / mo**	**~$135 – $235 / mo**	**Highly cost-effective infrastructure for school deployment.**

15. Philippine Data Privacy Act of 2012 (RA 10173) & Terms and Conditions

The ARALert system processes **Sensitive Personal Information (SPI) of Minors** (Grade 7/8 students under 18 years old: LRN, academic risk status, quarterly grades, parental contact numbers). Under Republic Act No. 10173:

- **Parental/Guardian Consent:** Processing minor data requires verified parental consent or school institutional mandate under DepEd guidelines.
- **Mandatory Privacy Statement Items:**
  - **Purpose:** Academic progress tracking, intervention alerts, and DepEd ARAL program monitoring.
  - **Data Retention:** Stored for the duration of student enrollment plus statutory retention (5 academic years).
  - **Data Sharing:** Strictly limited to authorized school personnel, assigned tutors, and DepEd division offices.
  - **Data Subject Rights:** Right to be informed, right to access, right to rectify (via Student Edit Requests), and right to erasure.
  - **Data Protection Officer (DPO):** Designated school DPO contact information.

Placement in the Application:

1. **First-Time Login Consent Modal:** Non-dismissible modal displaying the Privacy Policy with an "Accept & Continue" button, storing `privacy_consent_accepted_at: timestamp` in the database.
2. **Login Screen Footer:** Direct links to `/terms-of-service` and `/privacy-policy` below the login buttons.

16. Root Cause Analysis of System Slowness & Performance Bottlenecks

> **Identified Latency Bottlenecks in the Codebase:**

1. **N+1 SQL Queries in `UserListSerializer`:** In `backend/api/users/views.py`, the user queryset does not use `select_related('assigned_tutor')`. Rendering 50 students executes **51 separate SQL queries** to resolve tutor names.
2. **Sequential `while (next)` Frontend Pagination:** In `academicService.ts`, fetching grades loops through every paginated page over the network before displaying data.
3. **Duplicate Profile Mount Requests:** `useAuth.tsx` fires a redundant GET request to `/api/profile/` on mount despite cached session data.
4. **SQLite Write Locks (Development):** With `USE_SQLITE=True`, SQLite locks the entire database file during write transactions (audit logging, attendance marks), stalling concurrent reads.
5. **Dashboard Aggregation Queries:** `UserStatsView` executes 12+ separate unindexed `.count()` queries on every admin dashboard load.

17. Code Optimization, Dead Code Removal & Refactoring Plan

- **Backend Query Optimization:** Add `select_related('assigned_tutor')` to `UserListCreateView` and combine `UserStatsView` counts into a single `User.objects.aggregate(...)` query.
- **Retire Dummy Middleware:** Remove the no-op `RoleMiddleware` in `backend/middleware/role_middleware.py`.
- **Frontend Data Layer:** Replace `while (next)` loops in `academicService.ts` with direct server-side page requests and rely on `sessionStorage` in `useAuth.tsx`.
- **Environment Hygiene:** Sanitize `frontend/.env` by removing all backend credentials.

- **Test Suite Maintenance:** Update legacy assertions in `backend/api/auth/tests.py` to reflect current Google OAuth and LRN authentication workflows.
